WT : EXPERIMENT 6

Topic: Interfacing of Wireless notice board.

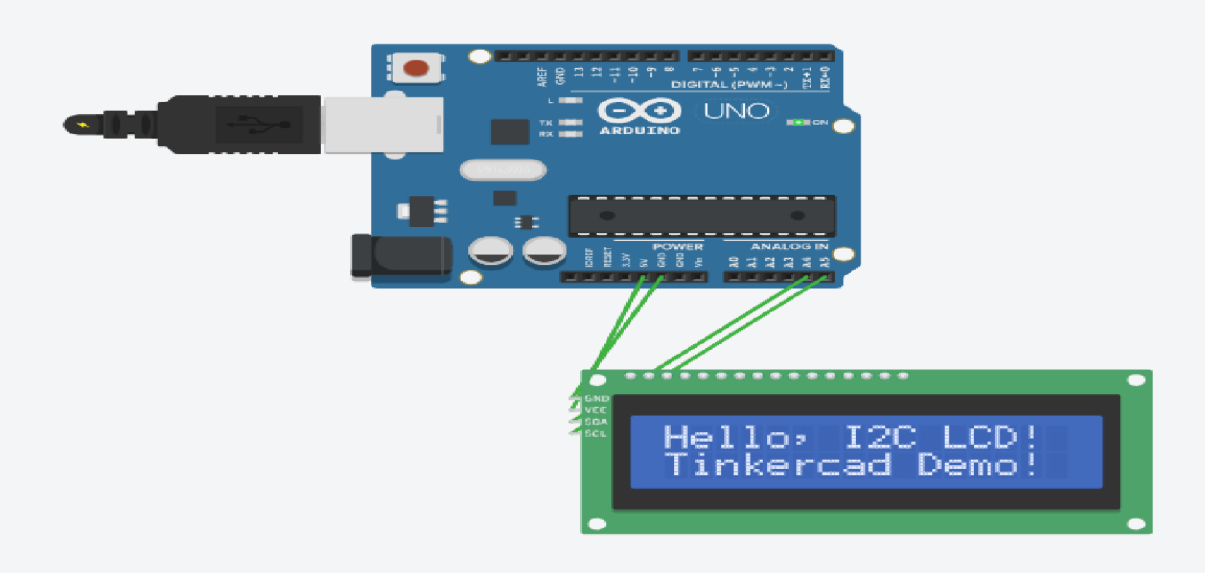
SR NO:	CLASS	NAME	ROLL NO
1.	D15A	Siddhant Sathe	50
2.	D15A	Arnav Sawant	52
3.	D15A	Pranav Titambe	60

Aim: Study of Arduino Nano and interfacing with external components for Wireless notice board with temperature sensor.

Introduction:

In modern educational and corporate environments, efficient and accurate attendance tracking is crucial. Traditional methods often involve manual entry, which can be time-consuming and prone to errors. To address these challenges, this project presents the development of a **Smart IoT-Based RFID Attendance System** utilizing the **NodeMCU ESP8266** microcontroller and **Google Sheets** for real-time data management.

This system leverages **Radio-Frequency Identification (RFID)** technology to enable quick and contactless identification of individuals. Each participant is issued a unique RFID tag, which, upon scanning, communicates with the NodeMCU ESP8266. The microcontroller processes the tag information and transmits it over Wi-Fi to update attendance records in a Google Sheets document instantaneously.



NodeMcu esp8266 to RFID reader

VCC	3.3V
GND	GND
SDA	D2
SCK	D5
MOSI	D7
MISO	D6
RST	D4
VCC	3.3V

NodeMCU 8266 to LCD pin

VCC	3.3V
GND	GND
SDA	D1
SCL	D2

NodeMCU to buzzer pin

Positive	D8
Negative	GND

Code for connection:

```
#include <SPI.h>
#include <MFRC522.h>
#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>
#include <WiFiClientSecureBearSSL.h>
#include <LiquidCrystal_I2C.h>
#define RST_PIN  D3
#define SS_PIN   D4
#define BUZZER   D8

MFRC522 mfrc522(SS_PIN, RST_PIN);
MFRC522::MIFARE_Key key;
MFRC522::StatusCode status;
```

```
int blockNum = 2;
byte bufferLen = 18;
byte readBlockData[18];

String card_holder_name;
const String sheet_url =
"https://script.google.com/macros/s/AKfycbzoyb0VXBUY8lcp4pCJ-4x_Q2dCXOlnfds7otEZUDt4EkmQeTUri
MAoRWSt1IMiqxcu/exec?name=";  //Enter Google Script URL

#define WIFI_SSID "Ghe hotspot"  //Enter WiFi Name
#define WIFI_PASSWORD "00000000"  //Enter WiFi Password

//Initialize the LCD display
LiquidCrystal_I2C lcd(0x27, 16, 4);


void setup()
{
    /* Initialize serial communications with the PC */
  Serial.begin(9600);
  //Serial.setDebugOutput(true);

  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("  Initializing  ");
  for (int a = 5; a <= 10; a++) {
    lcd.setCursor(a, 1);
    lcd.print(".");
    delay(500);
  }

   //WiFi Connectivity
  Serial.println();
  Serial.print("Connecting to AP");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED){
    Serial.print(".");
    delay(200);
  }
  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  Serial.println();
    /* Set BUZZER as OUTPUT */
  pinMode(BUZZER, OUTPUT);
    /* Initialize SPI bus */
  SPI.begin();

}
 void loop()
{
```

```
  lcd.clear();
lcd.setCursor(0, 0);
lcd.print(" Scan your Card ");
/* Initialize MFRC522 Module */
mfrc522.PCD_Init();
/* Look for new cards */
/* Reset the loop if no new card is present on RC522 Reader */
if ( ! mfrc522.PICC_IsNewCardPresent()) {return;}
/* Select one of the cards */
if ( ! mfrc522.PICC_ReadCardSerial()) {return;}
/* Read data from the same block */
//-------------------------------------------------
Serial.println();
Serial.println(F("Reading last data from RFID..."));
ReadDataFromBlock(blockNum, readBlockData);
Serial.println();
Serial.print(F("Last data in RFID:"));
Serial.print(blockNum);
Serial.print(F(" --> "));
for (int j=0 ; j<16 ; j++)
{
  Serial.write(readBlockData[j]);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Hey " + String((char*)readBlockData) + "!");
}
Serial.println();
digitalWrite(BUZZER, HIGH);
delay(200);
digitalWrite(BUZZER, LOW);
delay(200);
digitalWrite(BUZZER, HIGH);
delay(200);
digitalWrite(BUZZER, LOW);
if (WiFi.status() == WL_CONNECTED) {
  std::unique_ptr<BearSSL::WiFiClientSecure>client(new BearSSL::WiFiClientSecure);
      client->setInsecure();
  card_holder_name = sheet_url + String((char*)readBlockData);
  card_holder_name.trim();
  Serial.println(card_holder_name);

  HTTPClient https;
  Serial.print(F("[HTTPS] begin...\n"));
        if (https.begin(*client, (String)card_holder_name)){

    // HTTP
    Serial.print(F("[HTTPS] GET...\n"));
    // start connection and send HTTP header
    int httpCode = https.GET();

    // httpCode will be negative on error
    if (httpCode > 0) {
      // HTTP header has been send and Server response header has been handled
      Serial.printf("[HTTPS] GET... code: %d\n", httpCode);
      // file found at server
  lcd.setCursor(0, 1);
  lcd.print(" Data Recorded ");
```

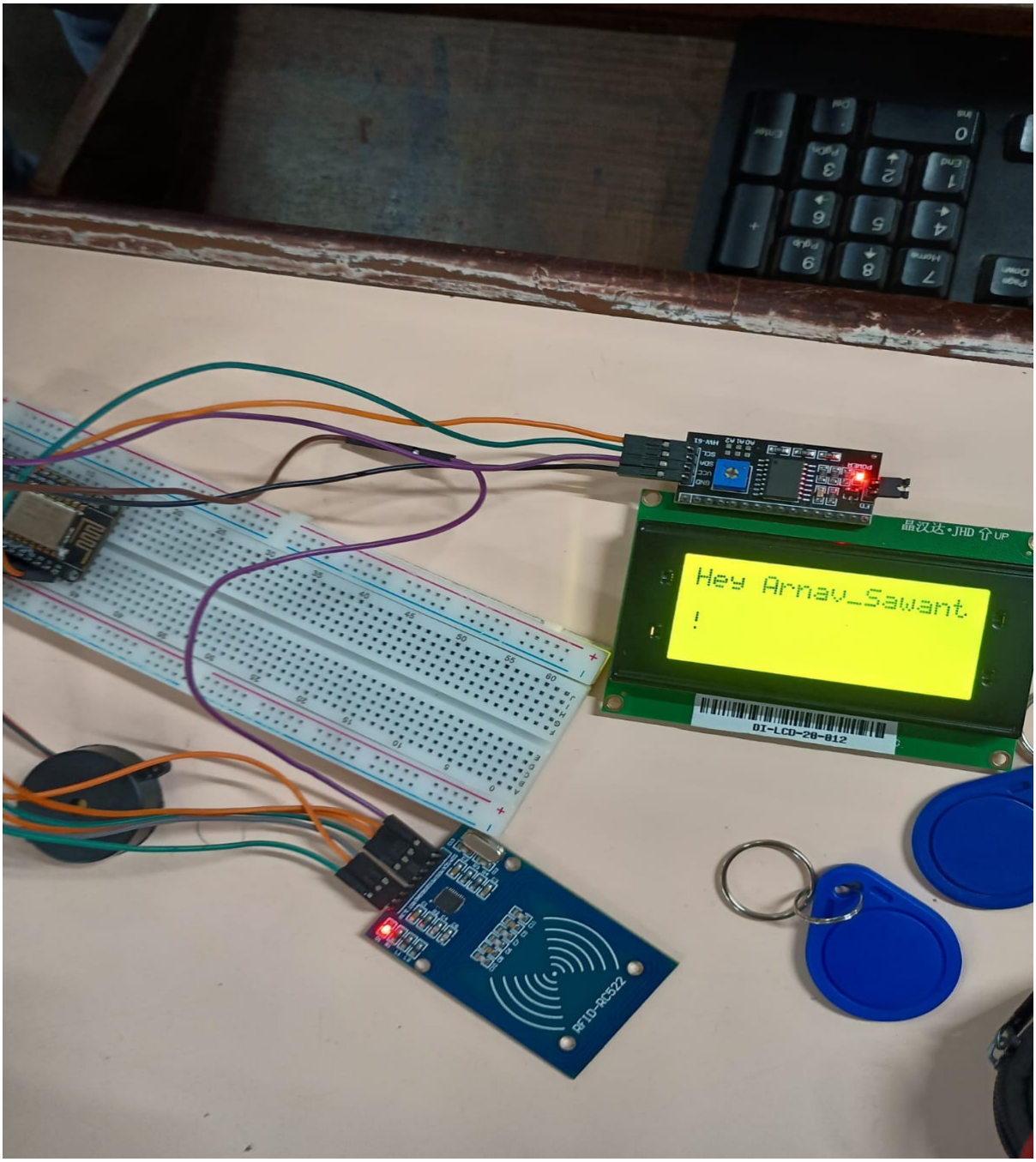
```
    delay(2000);
      }

      else
      {Serial.printf("[HTTPS] GET... failed, error: %s\n", https.errorToString(httpCode).c_str());}

      https.end();
      delay(1000);
    }
    else {
      Serial.printf("[HTTPS} Unable to connect\n");
    }
void ReadDataFromBlock(int blockNum, byte readBlockData[])
{
    for (byte i = 0; i < 6; i++) {
    key.keyByte[i] = 0xFF;
  }

  /* Authenticating the desired data block for Read access using Key A */
  status = mfrc522.PCD_Authenticate(MFRC522::PICC_CMD_MF_AUTH_KEY_A, blockNum, &key,
&(mfrc522.uid));

  if (status != MFRC522::STATUS_OK){
     Serial.print("Authentication failed for Read: ");
     Serial.println(mfrc522.GetStatusCodeName(status));
     return;
  }
    else {
    Serial.println("Authentication success");
  }
    /* Reading data from the Block */
  status = mfrc522.MIFARE_Read(blockNum, readBlockData, &bufferLen);
  if (status != MFRC522::STATUS_OK) {
    Serial.print("Reading failed: ");
    Serial.println(mfrc522.GetStatusCodeName(status));
    return;
  }

  else {
    Serial.println("Block was read successfully");
  }
  }
```

Libraries Used:

1. <Wire.h>
   The Wire library is used for I2C (Inter-Integrated Circuit) communication. It facilitates communication between the Arduino and I2C-compatible devices such as sensors, displays, and other microcontrollers.

2. <LiquidCrystal_I2C.h>
   This library provides functions to interface with Liquid Crystal Displays (LCDs) that use the I2C communication protocol. It simplifies operations such as initializing the display, setting the cursor position, and printing text.

Implementation:



Conclusion:

The Smart IoT-Based RFID Attendance System offers a modern, efficient solution to attendance tracking by minimizing human error and reducing administrative overhead. By integrating RFID technology with the NodeMCU ESP8266 and Google Sheets, the system ensures **real-time, accurate, and contactless attendance logging**, making it highly suitable for both educational and corporate environments.

· **Automation and real-time updates** significantly enhance the accuracy and efficiency of attendance management compared to traditional manual methods.

· **IoT integration using affordable hardware** like the NodeMCU ESP8266 makes smart attendance systems accessible, scalable, and easy to implement in various institutions.